Hardware-Portable, Persona-Preserving Optimisation of the Unchessed UCI Engine

Manus AI
Independent Engine Systems Research
Unchessed Heavy Optimisation
Research branch: manus/research-facilities

***Abstract*—This paper reports an isolated optimisation sub-project for the Unchessed UCI chess engine. The work preserves five adaptive personas—Full, Match, Clinch, Punish, and Defend—while improving protocol correctness, low-time stability, stale neural-hint resistance, and broad Intel/AMD portability. The implementation removes an unconditional `target-cpu=native` build setting, adds shared-L3-aware transposition-table defaults, enables smoothed persona transitions by default, and adds a post-transition cooldown that prevents non-emergency reversals. Rust 1.98.1 validation produced 124 passing workspace tests, successful deep perft, UCI smoke success, and portable and x86-64-v3 release builds. A one-thread benchmark across four positions and five hash sizes measured a 0.52–6.68 percent x86-64-v3 mean-NPS advantage on the test virtual machine. The paper also formalizes the implemented internal neural root-hint firewall and separates it from the future external Lc0 UCI provider design. Alpha-beta remains authoritative: neural evidence may order legal root moves, but cannot remove moves, alter safety semantics, or select a persona move directly.**

***Index Terms*—UCI chess engine, persona stability, NNUE, root priors, cache-aware transposition table, Lc0 verification**

I. Introduction

Adaptive chess engines have two competing requirements. Objective search must remain tactically sound, while humanised play must express a controlled style rather than random weakening. Unchessed adds a third requirement: its Full, Match, Clinch, Punish, and Defend personas must remain distinct and operational under changing opponent evidence, clock pressure, and opening-book behavior. This paper studies how to improve performance and stability without allowing external neural evidence to bypass that contract.

Modern engines provide complementary techniques rather than interchangeable scores. Stockfish combines selective alpha-beta search with NNUE evaluation [[1]; [2]]. Leela Chess Zero (Lc0) uses a neural policy/value system in a PUCT-style search [[3]; [4]]. Maia models human move behavior rather than objective best play [[5]; [6]]. AlphaZero established a public policy/value self-play framework but did not publish all production implementation details [[7]]. The present work therefore adopts interfaces and invariants, not unverified claims of reproducing any proprietary system.

The contributions are fourfold:

1) A portable build policy that distinguishes baseline x86-64, x86-64-v3, and exact-host artifacts.
2) A cache-aware default transposition-table policy responsive to shared L3 capacity.
3) A strengthened persona state machine with default exponential smoothing, emergency overrides, transition dwell, and a post-transition cooldown.
4) An explicit analysis of the root-prior firewall, including legal-move anchoring, finite-score filtering, stale-key prevention, and alpha-beta authority.

TABLE I
Protected persona invariants.

Persona	Invariant
Full	Use the strongest completed alpha-beta result.
Match	Respect target Elo and human-plausible candidate sampling.
Clinch	Use bounded reply-gap probes only when the budget allows.
Punish	Never decline a found mate; prefer forcing conversion when ahead.
Defend	Prefer maximum resistance and do not repel safe draws.

II. Protected persona contract

The persona layer is not a cosmetic move picker. It is a stateful policy controller over completed search lines. Full selects the strongest completed line. Match samples from a calibrated candidate distribution at a target Elo. Clinch probes reply gaps and favors narrow-path opportunities. Punish prioritizes mates, forcing captures, and checks. Defend selects maximum resistance. Opponent modelling estimates skill from weighted centipawn loss and clock evidence. The troll book is risk-tiered and has a shallow refutation guard. Draw contempt depends on the active objective.

A neural provider must therefore be subordinate to the contract. It may provide evidence about move order or candidate plausibility. It may not silently change the active persona, add a troll line, disable a safety veto, or turn a human-policy score into objective centipawns.

III. Stability methodology

A. Opponent evidence

The opponent model begins with a broad prior and updates it using a logarithmic centipawn-loss-to-Elo curve. Difficulty weighting discounts book, forced, and trivial moves. Clock suspicion requires strong moves, real choice, and repeated evidence; opening observations are discounted in the experi-

mental detector. A single clean opening sequence is therefore not sufficient to force Full mode.

The implementation corrects two evidence-integrity defects. A book observation uses the move's actual ply rather than the final game ply. When pending observations are processed in a batch, the opponent clock signal is applied only to the newest observation. These changes prevent historical moves from receiving current-game timing evidence.

B. Transition filtering

The persona state uses an exponential moving average (EMA) with coefficient 0.35 on the newest completed search evaluation. The first evaluation seeds the filter without counting as a transition vote. A candidate mode requires two agreeing updates. Emergencies bypass dwell: a suspected engine can force Full, a severe raw or smoothed collapse can force Defend, and a fresh verified opponent blunder while ahead can force Punish.

The new stability control adds a two-update cooldown after a non-emergency transition. During cooldown, a conflicting proposal is recorded as the candidate but cannot reverse the active mode. Emergency Full, Defend, and Punish transitions always clear the cooldown and take effect immediately. Opt-in telemetry now reports `candidate`, `dwell`, `cooldown`, and the emergency reason, making mode decisions auditable without feeding telemetry back into selection.

Persona smoothing is now enabled by default. The explicit `PersonaSmooth` UCI option remains available for controlled ablations. This is a behavioral safety default: it suppresses accidental Clinch or Match/Punish oscillation caused by noisy iterative-deepening evaluations while preserving rapid responses to tactical emergencies.

C. Low-time policy

Before opponent observation, book verification, or persona probing, the UCI worker computes a low-time gate. The gate activates for less than ten seconds of the relevant clock, a computed hard budget below one second, explicit node limits, depth at most two, or movetime below one second. In that state, side searches are skipped and the move search receives the available budget. This prevents adaptive work from consuming the move's deadline.

The same principle governs a future Lc0 provider. Provider startup, queueing, inference, IPC, stop, and drain costs must be subtracted from the hard deadline before a request is spawned. Critical and Emergency tiers do not wait for a cold neural process.

IV. Hardware portability and cache policy

The optimisation copy previously forced `target-cpu=native`, which can emit instructions unavailable on another deployment machine. The revised policy produces three deliberate tiers.

Runtime-specialised kernels must use feature detection and retain a scalar oracle. AVX-512 is not assumed. Core Ultra 9 285H measurements should prefer AVX2 or AVX-VNNI where empirically justified, while Ryzen 7 7730U systems should use AVX2 when available and otherwise fall back cleanly. Quantised NNUE and VNNI require model-format and numerical-parity validation; they are not compiler-flag-only optimisations.

TABLE II
Build tiers for broad CPU coverage.

Tier	Build policy	Deployment
Portable	No host-specific flags; scalar and portable paths remain valid.	Legacy x86-64, VMs, broad Intel/AMD support.
Modern	`-C target-cpu=x86-64-v3` in a separately labelled artifact.	AVX2/FMA/BMI2-class consumer and server CPUs.
Host-tuned	`-C target-cpu=native` only for a pinned machine.	Exact deployment CPU after local benchmark.

The transposition table is a random-access structure. A table that greatly exceeds shared L3 can increase memory latency and displace evaluator state. The default `Hash` now reads Linux shared-L3 metadata from `/sys/devices/system/cpu/cpu0/cache/index3/size`, targets half of L3, and clamps the result to 4–128 MiB. Explicit UCI `Hash` values override automatic sizing. The current VM reports 36,608 KiB L3 and advertises a 17 MiB automatic default. The target examples are approximately 8 MiB for a 16 MiB-L3 Ryzen 7 7730U and 12 MiB for a 24 MiB-L3 Core Ultra 9 285H.

V. Neural root-prior firewall

A. Implemented internal path

The internal `search::go_with_root_hints` path begins with Unchessed's own legal move generator. If `searchmoves` is present, it intersects that legal list with the requested UCI strings. It constructs a root record for every remaining legal move. A supplied hint is used only when its move matches one of those legal moves and its `policy_score` is finite. Missing, NaN, and infinite scores become unusable ordering values. The hint list never becomes the legal move list.

The core operation is equivalent to:

```
let roots = legal(pos).iter().map(|&mv|
RootMove {
    mv,
    score: -MATE,
    policy_hint: hints.iter()
        .find(|h| h.mv == mv &&
h.policy_score.is_finite())
        .map(|h| h.policy_score)
        .unwrap_or(f32::NEG_INFINITY),
    ..
}).collect();
```

The search remains responsible for completed scores, bounds, mate handling, and final lines. Existing tests verify that stale or non-finite hints cannot remove legal moves, that hints cannot suppress forced mates, and that check positions retain only legal root moves. The Aegis cache key additionally binds en-passant state and halfmove buckets, preventing a

neural result for a superficially identical hash from crossing rule-state boundaries.

B. Security safeguards

The firewall is a semantic input boundary rather than a cryptographic security boundary. Its safeguards are:

1) **Authority separation:** alpha-beta owns final move validity and score completion.

+Legal-set anchoring: external candidates are intersected with generated legal moves. **+Finite-value filtering:** NaN and infinity cannot poison ordering or comparisons. **+Identity binding:** position hash, legal-action fingerprint, model hash, schema version, en-passant state, and halfmove bucket belong in an external-provider key. **+Deadline binding:** late results are rejected by request token and deadline. **+Fail-closed behavior:** any malformed provider reply falls back to ordinary search. **+Protocol isolation:** a future Lc0 child process must drain stdout and stderr, accept only a matching `bestmove` terminal event, and validate that move against Unchessed's legal generator.

These safeguards protect against stale outputs, malformed vectors, illegal moves, score poisoning, and timing races. They do not prove that a neural model is strategically correct. Model quality remains an empirical question.

C. External Lc0 verification design

The future Lc0 provider will pin the binary, network SHA-256, backend, driver/runtime, and UCI options. It will perform `uci`/`uciok` and `isready`/`readyok` handshakes, send complete position history, use one active search token, and stop-and-drain on cancellation. Lc0 visits, value, and policy are recorded as provider-specific evidence. They are not averaged directly with Stockfish centipawns or Maia WDL.

In Normal mode, a warmed Lc0 result may verify or reorder a candidate if the measured latency fits. In Fast mode, only one bounded request is allowed. In Critical and Emergency modes, a cold or uncertain provider is disabled. Lc0 cannot alter persona state, contempt, troll eligibility, or the final selection authority.

VI. Experimental evaluation

A. Validation suite

Rust 1.98.1 and the native linker were installed in the sandbox. The workspace validation passed 124 tests, with six ignored tests, and deep perft passed. UCI smoke tests found a legal start-position move and the required back-rank mate. Portable and x86-64-v3 release builds compiled successfully.

B. Portable versus x86-64-v3 benchmark

The benchmark used one search thread, disabled adaptive shortcuts and the opening book, tested four positions, and swept Hash values of 4, 8, 16, 32, and 64 MiB. Each case received a 500,000-node maximum and a two-second graceful UCI window. Mean NPS and maximum resident memory were recorded.

The v3 build was 0.52–6.68 percent faster in mean NPS. The advantage was not monotonic in Hash size. The 8–32 MiB range was generally favorable, while 64 MiB increased memory without a consistent throughput benefit. Portable and v3 RSS was effectively equal because code generation does not change the TT allocation layout.

TABLE III
One-thread NPS and memory scaling on the test VM.

Build	Hash	Mean NPS	v3 delta	Mean RSS
Portable	4 MiB	1,823,284	–	23.7 MiB
v3	4 MiB	1,832,842	+0.52%	23.7 MiB
Portable	8 MiB	1,933,064	–	27.7 MiB
v3	8 MiB	1,986,948	+2.79%	27.7 MiB
Portable	16 MiB	1,871,675	–	35.7 MiB
v3	16 MiB	1,971,977	+5.36%	35.7 MiB
Portable	32 MiB	1,908,441	–	51.6 MiB
v3	32 MiB	1,924,170	+0.82%	51.6 MiB
Portable	64 MiB	1,820,084	–	83.7 MiB
v3	64 MiB	1,941,687	+6.68%	83.7 MiB

These are measurements of one virtual machine, not claims about all Intel or AMD systems. They do not measure hybrid-core placement, thermal throttling, NUMA, AVX-VNNI, or a production NNUE file. The raw data and harness are retained for reproduction.

VII. Discussion and limitations

The persona changes are intentionally conservative. Smoothing and cooldown reduce flapping, but they do not calibrate the Elo model or prove human-likeness. A future study should use rating-stratified human games and evaluate move-match likelihood, blunder-frequency calibration, mode-switch frequency, and low-time forfeits. Each tree-changing search feature should be isolated and evaluated with fixed-node correctness tests followed by a preregistered SPRT.

The root-prior firewall is robust against malformed or stale inputs at the internal interface, but the external Lc0 process adapter is not yet part of the production code. The paper distinguishes implemented behavior from planned integration to avoid overstating completion. A typed provider trait, supervised child-process state machine, model manifest, and deadline planner are the next implementation stage.

The hardware default is Linux-specific and deliberately conservative. Portable cache discovery, per-core versus shared-cache topology, memory-bandwidth measurement, and evaluator working-set profiling remain future work. The benchmark's NPS values should not be extrapolated to the Core Ultra 9 285H, Ryzen 7 7730U, legacy CPUs, or data-center CPUs without local measurements.

VIII. Conclusion

The sub-project improves Unchessed by making persona transitions more stable, preserving emergency tactical behavior, preventing stale neural state, and adapting compilation and Hash defaults to heterogeneous hardware. The key architectural result is an authority boundary: external neural and engine evidence may influence ordering or verification only after identity, legality, finiteness, timing, and protocol checks.

Completed alpha-beta search and the active persona remain authoritative.

The measured x86-64-v3 advantage on the current VM is real but modest. The more important portability result is that modern and legacy deployments can use deliberately separated artifacts rather than a single unsafe host-specific binary. The resulting foundation supports future Lc0, Maia, AlphaZero-inspired, and NNUE work without allowing those additions to misfire the protected Unchessed behaviors.

References

[1] S. developers, "Official Stockfish repository." [Online]. Available: https://github.com/official-stockfish/Stockfish

[2] S. developers, "Stockfish NNUE technical documentation." [Online]. Available: https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html

[3] L. C. Z. developers, "Leela Chess Zero developer overview." [Online]. Available: https://lczero.org/dev/overview/

[4] L. C. Z. developers, "Technical explanation of Leela Chess Zero." [Online]. Available: https://lczero.org/dev/wiki/technical-explanation-of-leela-chess-zero/

[5] R. McIlroy-Young, S. Sen, J. Kleinberg, and A. Anderson, "Aligning Superhuman AI with Human Behavior: Chess as a Model System," *Proceedings of the 26th ACM SIGKDD International Conference*, 2020, doi: 10.1145/3394486.3403219.

[6] R. McIlroy-Young and others, "Maia-2: A Unified Model for Human-Aware Chess," 2024, [Online]. Available: https://arxiv.org/abs/2409.20553

[7] D. Silver and others, "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm," 2017, [Online]. Available: https://arxiv.org/abs/1712.01815

References